

# **Rust, CUDA and Verus**

## A PoC on Rust-CUDA Verifiable Kernels

---

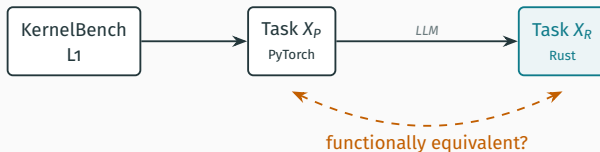
Mahadi Hasan   Anamul Hoque Emtiaj

October 6, 2026

UIUC SRSE 2026

# From PyTorch to Rust

**Source of the CUDA kernels:** KernelBench Level 1 tasks (for now).



**Goal.** Guarantee that the translated task computes the same function as the original.

$$X_P, X_R : \mathcal{X} \rightarrow \mathcal{Y} \quad \forall x \in \mathcal{X}. \llbracket X_P \rrbracket(x) = \llbracket X_R \rrbracket(x)$$

# Two KernelBench L1 tasks

## 19\_ReLU.py

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.relu(x)

batch_size = 4096
dim = 393216
def get_inputs():
    x = torch.rand(batch_size, dim)
    return [x]
def get_init_inputs():
    return []
```

**What it computes.** Replaces every negative entry with zero; the shape is unchanged.

$$x \in \mathbb{R}^{4096 \times 393216}, \quad y_{ij} = \max(0, x_{ij})$$

## 47\_Sum\_reduction\_over\_a\_dimension.py

```
class Model(nn.Module):
    def __init__(self, dim: int):
        super(Model, self).__init__()
        self.dim = dim
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.sum(x, dim=self.dim, keepdim=True)

batch_size = 128
dim1 = 4096
dim2 = 4095
reduce_dim = 1
def get_inputs():
    x = torch.rand(batch_size, dim1, dim2)
    return [x]
def get_init_inputs():
    return [reduce_dim]
```

**What it computes.** Adds up the entries along the middle axis, keeping it as size 1.

$$x \in \mathbb{R}^{128 \times 4096 \times 4095}, \quad y_{b,o,k} = \sum_{j=0}^{4095} x_{b,j,k}$$

Source: KernelBench @ 423217d, KernelBench/level1/; docstrings and imports omitted.

# Faithful spec extraction

Initially, Mahadi worked on how to *faithfully* extract a specification from a PyTorch source. The Python file is the specification, so the job is **denotation**: what function does it compute?

## Task 47 (PyTorch)

```
return torch.sum(x, dim=1, keepdim=True)
```

### Question about I/O

- 1 How is the input x held?
- 2 Which inputs feed output cell r?
- 3 What is done with them?
- 4 What type is an output value?
- 5 How many outputs, and what is each?

### Spec it creates

```
⇒ x: Seq<i32>, rows: int, cols: int // flat, row-major
⇒ row(x, r, cols) = x.subrange(r*cols, r*cols + cols)
⇒ fold_sum(s) = if s.len() == 0 { 0 } else { s[0] + fold_sum(s.skip(1)) }
⇒ fold_sum(s: Seq<i32>) -> int // overflow cannot hide in the spec
⇒ torch_sum_dim1(x, rows, cols) = Seq::new(rows, |r| fold_sum(row(x, r, cols)))
```

**Insights.** Questions about I/O reveal the necessary specs. However, faithfulness up to this point remains a spec-testing problem; human audit remains the final standard.

# Verus-KernelBench: the plan (Anamul)

The same idea, scaled up: a **faithful spec** and a **verified Rust GPU kernel** for every KernelBench Level 1 task, and then a way to **measure LLMs** on it.

## 1 Benchmark

POC

All **100** KernelBench L1 tasks. Each one becomes an *entry*: the PyTorch program, its Verus spec, and a Rust GPU kernel proved to meet that spec.

## 2 Harness

POC

A human-agent pipeline that **builds** the benchmark, and that **evaluates** LLMs by running their output through the same checks.

## 3 Evaluation

NEXT

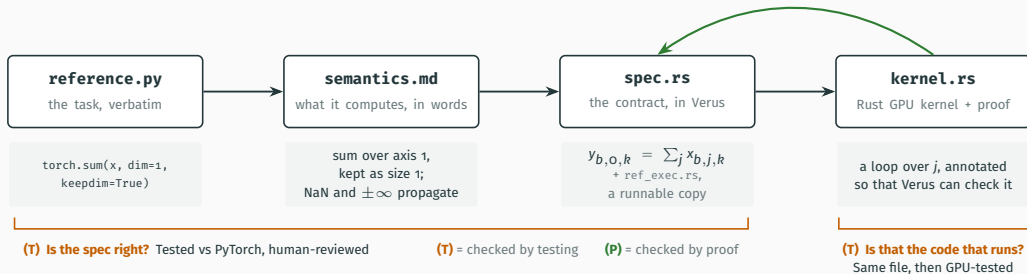
Measure how well different LLMs generate **verifiable** Rust kernels, given three levels of help.

**Where we are.** A proof of concept of 1 and 2 exists. This talk presents the design, and the plan to complete all 100 tasks and run 3.

# Current Pipeline: Task 47 (Anamul)

One benchmark entry is four files. Running example: Task 47 (page 3), the sum over one axis.

(P) Verus proves: the kernel meets the spec, for every input



semantics.md records:

Operator & defaults

Dtypes

Layout

Preconditions

Parameters & state

Special values

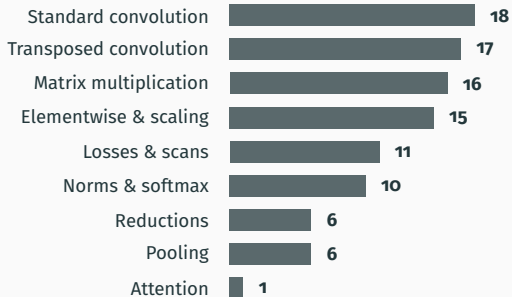
Reduction order

Domains

**Special values:** corner cases such as NaN,  $\pm\infty$  and  $-0.0$ , which KernelBench's random input generator never produces.  
**Reduction order:** the order of the additions, which changes a float result.

# Humans define, agents scale (Anamul)

## 100 KernelBench L1 tasks, grouped into 9 families



### Human defines

- 1 Group the 100 tasks into families by shared structure.
- 2 Document each family: its characteristics and how to build it.
- 3 Verify one *pilot* entry per family by hand.

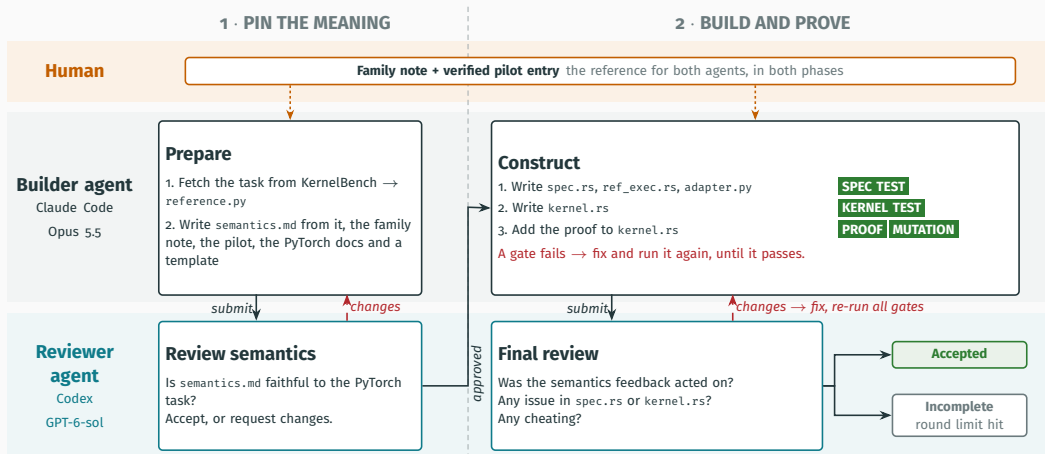
↓ family note + pilot entry

### Agents scale

Build every other task in the family, with the family note and the pilot entry as the reference (next slide).

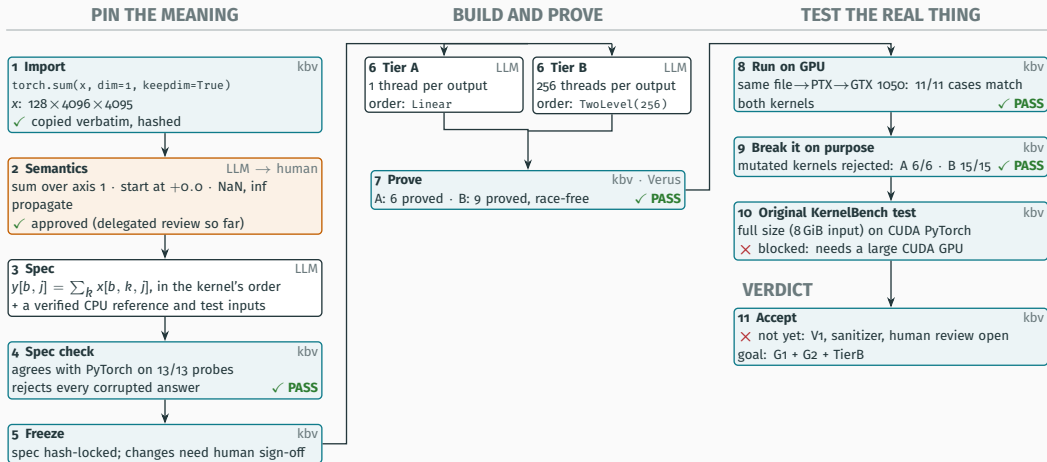
**Why families.** Human effort grows with the number of families (9), not with the number of tasks (100).

# The construction loop (Anamul)



**Loop:** until the reviewer accepts, or the entry is flagged *Incomplete* after a fixed number of rounds. **Independent review:** builder and reviewer are different agents, from different vendors.

# Task 47: End to End



# Four gates: why trust an entry (Anamul)

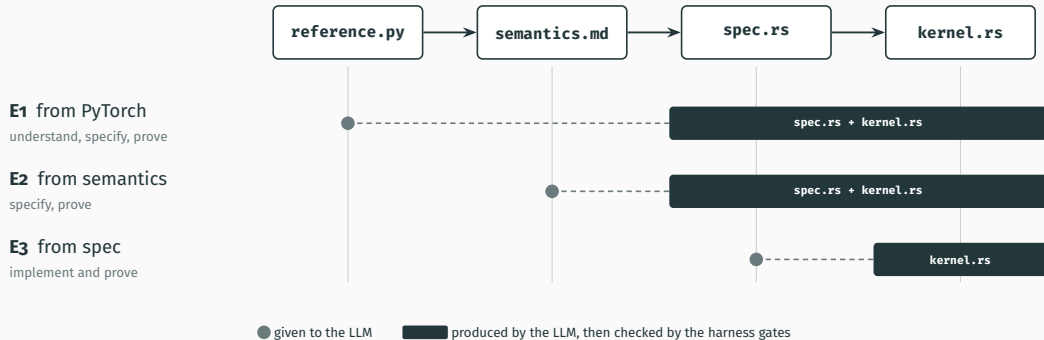
Inside *Construct*, every entry must pass four automatic gates run by the harness. A failed gate goes back to the builder, which fixes the files and runs the gate again.

| Gate                                | Question                                                               | How                                                                                                                                                                                               | Passes when                                      |
|-------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>1 Spec test</b><br><b>TEST</b>   | Does <code>spec.rs</code> say what PyTorch computes?                   | Verus checks the spec on concrete cases. <b>Positive:</b> small-integer inputs (exact output), plus corner and tolerance cases from <code>semantics.md</code> . <b>Negative:</b> mutated outputs. | every positive holds, every negative is rejected |
| <b>2 Kernel test</b><br><b>TEST</b> | Does <code>kernel.rs</code> compute the right numbers?                 | Run it on KernelBench's own test inputs and compare with PyTorch.                                                                                                                                 | results match, within tolerance                  |
| <b>3 Proof</b><br><b>PROOF</b>      | Does <code>kernel.rs</code> meet <code>spec.rs</code> for every input? | Verus checks the proof in <code>kernel.rs</code> against the spec.                                                                                                                                | Verus accepts                                    |
| <b>4 Mutation</b><br><b>PROOF</b>   | Is the proof meaningful, not vacuous?                                  | Break <code>kernel.rs</code> on purpose (mutants) and run Verus again.                                                                                                                            | every mutant is rejected                         |

**Together.** Tests tie the spec to PyTorch on known cases; the proof then covers every input against that spec; mutation shows that the proof actually constrains the kernel.

# Evaluating LLMs: three levels of help (Anamul)

An entry is a chain of files. Each evaluation task gives the LLM one file and asks it to produce the spec and the kernel.



**Why three levels.** Comparing E1, E2 and E3 shows where an LLM breaks down: understanding the PyTorch task, writing a faithful spec, or building a kernel that Verus accepts.

# Current Landscape

## Accountability layer

Mahadi takes Anamul's repo as a **read-only artifact** and runs his own **independent review**: the accountability layer for the KernelBench L1 tasks.

## But

Although SOTA papers use them too, KernelBench tasks were created to test **performance**, not **correctness**.

## So: VerCors CUDA case studies

A set of CUDA tasks originally developed for **formal verification**. Mahadi is working on them to find which tasks make the most sense from a **GPU programmer's perspective** of verified kernels.

# Publication Target

Reframing the paper for the following venues:

|              | Venue                | Deadline    | Focus                                                                                                                                               | Confidence |
|--------------|----------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| TARGET       | LLM4Code @ ICSE 2027 | 13 Nov 2026 | Non-archival option (tentative)                                                                                                                     | High       |
| LATER TARGET | CAV 2027             | 20 Jan 2027 | <b>Verification.</b> Propose an agent-based verification technique: the block-synchronous GPU model, its soundness argument, and the order library. | Medium     |
| LATER TARGET | ISSTA 2027           | 11 Jan 2027 | <b>Testing.</b> How weak KernelBench’s oracle is, and how well spec validation catches defects.                                                     | Medium     |

| Criteria          | Confidence             |
|-------------------|------------------------|
| Spec faithfulness | High                   |
| Technical rigor   | Low (Nov)    Mid (Jan) |

# Review of the current dataset

**Technical rigor** is what Mahadi thinks the current dataset lacks, for example:

- Verus does not support float arithmetic, but Mahadi finds that Anamul's current PoC replaces float addition with a trusted `#[verifier::external_body]` function.
- SOTA papers like *Kuiper* do not do this; rather, they show the approximation explicitly.
- The dataset's Rust code is generated by LLMs without the frameworks real GPU programmers would use in future work, i.e. `cuda-oxide` or `cuTile-Rust`.
- In Mahadi's own findings, using `cuda-oxide` or `cuTile-Rust` with in-place Verus annotations, which real GPU communities would actually use, is more time-consuming and less likely to fit the Nov deadline.

**Counter-argument.** These are engineering nitpicks that would not matter significantly. The pipeline does the intended job, and it is better to focus on the research contributions instead.

# Blockers and Literature

## Blockers

- Anamul has a GTX 1050 with 2 GB VRAM, which is insufficient for many KernelBench L1 tasks.
- Mahadi got access to an RTX 3070 machine a couple of days ago. Claude estimates that about 60 of the 100 L1 tasks can be tested on it.
- **Ask:** access to a modern Nvidia GPU with 24 GB VRAM, like an RTX 4090.

## Literature

- *Kuiper: GPU Kernel Verification in Pulse* does almost the same work we are doing, so their dataset is helpful to cross-check against our Verus port.
- *ProofWright: Towards Agentic Formal Verification of CUDA* claims it will open-source its artifact MLRocq, which converts PyTorch → Rocq “deterministically”, but has not yet. Early access would potentially make our work easier and give another base to cross-check against.